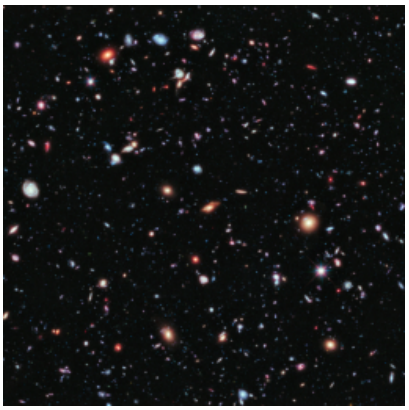
KALINGA INSTITUTE OF INDUSTRIAL TECHNOLOGY

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

Kalinga Institute of Industrial Technology (KIIT)

Exploring the Deep Universe with Digital Image Processing

Enhancement, Restoration, Compression and Analysis of the Hubble Deep Field and Lunar Surface Images



Test image: Hubble eXtreme Deep Field (NASA/ESA), 256 × 256 crop

Subject: Image Processing and Applications (IPA)

Semester: 5th

Submitted by:

Soumya Ranjan Sahoo | Roll No.: 241551053

Saikrishna Pal | Roll No.: 24155951

Section: CSE-30

Abstract

This project builds a complete digital image processing pipeline in Python and applies it to two famous real images: the Hubble eXtreme Deep Field, one of the deepest views of the universe ever captured by NASA/ESA, and a photograph of the lunar surface. The pipeline covers sampling and quantization, histogram-based and spatial enhancement, frequency-domain filtering, image restoration, colour processing, a JPEG-style compression codec, edge detection, segmentation, morphology and feature extraction. All core algorithms are implemented from scratch in NumPy; libraries are used only to cross-check results and for a few standard primitives (CLAHE, Canny, HSV conversion, region properties, K-means).

The main result is a working lossy codec built from the DCT, quantization, zigzag scanning, run-length coding and separate DC/AC Huffman tables. On the 256 × 256 colour Hubble image at quality 50 it reaches a compression ratio of 25.5:1 at 30.5 dB PSNR, against 23.6:1 for Pillow's default JPEG encoder and 25.2:1 for Pillow with optimised Huffman tables. Further experiments show that the median filter beats linear filters on impulse noise by 10.3 dB while a Gaussian filter is better on Gaussian noise, that regularised and truncated inverse filters restore a blurred, noisy image while the plain inverse filter fails, that a notch filter lifts periodic-noise PSNR from 20.3 to 48.8 dB, and that Canny edges survive heavy compression. An application study counts 121 bright objects (galaxies and stars) in the Hubble field with a classical thresholding pipeline.

The report compares the codec with a reference JPEG encoder and the classical filters with each other using PSNR, SSIM, compression ratio, accuracy, sensitivity, specificity, F1 score and Cohen's kappa, and lists the novelty, limitations and future scope of the work.

Keywords: image enhancement, image restoration, Wiener filter, DCT, JPEG, Huffman coding, rate-distortion, edge detection, Hubble Deep Field, PSNR, SSIM.

Table of Contents

Abstract 2

Table of Contents 3

1. Introduction 5

1.1 Objectives . . . 5

1.2 Tools and test data . . . 5

1.3 Organisation of the report . . . 5

1.4 Why these images? . . . 5

1.5 Novelty and contribution . . . 5

2. Proposed Methodology 6

2.1 Data used and its source . . . 6

2.2 Digital image fundamentals (Unit I) . . . 6

2.3 Spatial-domain enhancement (Unit II) . . . 6

2.4 Frequency-domain processing (Unit III) . . . 7

2.5 Image restoration (Unit IV) . . . 7

2.6 Colour processing and compression (Unit V) . . . 7

2.7 Morphology, edges and segmentation (Unit VI) . . . 7

2.8 Feature extraction (Unit VII) . . . 7

2.9 Evaluation metrics . . . 8

2.10 Implementation . . . 8

3. Experiment Segment 8

Part 1: Sampling and quantization . . . 8

Part 2: Spatial enhancement and denoising . . . 8

Part 3: Frequency-domain filtering . . . 10

Part 4: Image restoration . . . 11

Part 5: Colour models and compression . . . 12

Part 6: Edges, segmentation and morphology . . . 14

Part 7: Feature extraction and application: counting objects in the Hubble field . . . 16

Part 8: Discussion . . . 17

4. Literature Referred or Reviewed 17

4.1 Enhancement . . . 17

4.2 Restoration . . . 17

4.3 Compression . . . 17

4.4 Segmentation, edges and features . . . 18

4.5 Quality metrics and the data . . . 18

4.6 Summary of the reviewed work . . . 18

5. Comparison with Others 18

5.1 Metrics used . . . 18

5.2 Compression: the codec against the Pillow JPEG encoder . . . 19

5.3 Denoising: median against linear filters 20
5.4 Restoration: inverse, Wiener and CLS filters 20
5.5 Edge preservation after compression (all metrics) 20
6. Conclusion 21
7. Future Work (Future Scope) 21
8. Code Link 21
9. Bibliography 22
10. References 22

1. Introduction

Digital image processing transforms images so that they look better, can be recovered from damage, take less space, or reveal information that a person or a program can use. These jobs are usually taught one at a time. Astronomy is a field where all of them are needed together: telescope images are faint and noisy, they are blurred by optics and atmosphere, they are enormous and must be compressed for transmission, and scientists finally want to detect and count objects in them. This project therefore connects the stages in a single pipeline: an image is enhanced, deliberately degraded and restored, compressed and decompressed, and finally analysed for edges, regions and objects, with every stage measured by the same metrics.

1.1 Objectives

- Implement the core algorithms of every unit of the course (digital image fundamentals, spatial enhancement, frequency-domain processing, restoration, colour and compression, morphology and segmentation, feature extraction) instead of only calling library functions.
- Evaluate every stage quantitatively with PSNR, SSIM, entropy, compression ratio, edge F1 score and confusion-matrix metrics.
- Build a JPEG-style codec and analyse its rate-distortion behaviour against a real JPEG encoder.
- Study how lossy compression affects a downstream task, edge detection, and show a practical application: counting objects in a Hubble deep-field image.

1.2 Tools and test data

The code is written in Python 3 with NumPy [18], SciPy, scikit-image [17], scikit-learn, Matplotlib and Pillow. The test images are the **Hubble eXtreme Deep Field** (colour, used for colour processing, compression and object counting) and a grayscale photograph of the **lunar surface** (used for enhancement, filtering, restoration and edges). Both are resized to 256 × 256. The whole project runs with one command in about 9 seconds and writes every figure and number used in this report to a results folder, so all results are reproducible.

1.3 Organisation of the report

Section 1.5 states the novelty and contribution. Section 2 describes the methodology and the source of all data. Section 3 presents the experiments in seven parts followed by a discussion. Section 4 reviews the literature. Section 5 compares the codec with a reference JPEG encoder and the classical filters with one another. Sections 6 and 7 give the conclusion and future scope, Section 8 the code link, followed by the bibliography and references.

1.4 Why these images?

The Hubble eXtreme Deep Field is an almost black sky sprinkled with thousands of galaxies of very different brightness and size. That makes it an unusual and demanding test image: most pixels carry almost no information, so a good codec should compress it very strongly, histogram methods are needed to reveal faint objects, and segmentation has a clear goal (find the objects). The lunar surface image is a smooth, low-contrast, textured scene with craters, which is the classical case for contrast enhancement and for studying blur and noise.

1.5 Novelty and contribution

The individual algorithms (histogram equalization, Wiener filtering, the DCT, Huffman coding) are classical. The contribution lies in how they are combined, measured and checked. Specifically:

- **One connected pipeline on astronomical images.** Enhancement, restoration, compression and analysis are evaluated on the same two images with one set of metrics, so the effect of one stage on the next can be observed rather than assumed.
- **A from-scratch JPEG-style codec with a verified, honest rate.** It uses DC difference coding, zigzag scanning, (run, size) coding and four optimised Huffman tables (DC and AC for luminance

and chrominance). The decoder is run on the real bit stream to prove losslessness of the entropy stage, and the reported rate includes an estimate of the quantization-table and Huffman-table (DHT) header bytes.

- **A three-way benchmark.** The codec is compared with Pillow's default JPEG encoder and with Pillow using per-image optimised Huffman tables at seven quality settings. This separates the gain that comes from per-image tables from the gain that comes from the codec itself.
- **A downstream-task study.** Canny edge maps after compression are compared with the original with accuracy, sensitivity, specificity, precision, F1, Cohen's kappa and a one-pixel-tolerance F1.
- **An honest restoration study.** Parameters of the Wiener, CLS and truncated inverse filters are tuned with the original image available (an oracle), so the numbers are the best each filter can do; a blind constant-K Wiener estimate is also reported and shown to be much worse.
- **A practical application.** Objects in the Hubble field are detected and counted with and without gamma enhancement, showing how enhancement changes what can be found.

2. Proposed Methodology

The methodology is a seven-stage pipeline in which each stage is a syllabus unit: (I) sampling and quantization, (II) spatial enhancement, (III) frequency-domain filtering, (IV) restoration, (V) colour processing and compression, (VI) morphology, edges and segmentation, and (VII) feature extraction. The output of one stage can be the input of the next: for example a decoded image is passed to the edge detector, and an enhanced image is passed to the object counter. Every stage is measured with the metrics of Section 2.9.

2.1 Data used and its source

No dataset was downloaded and no data were collected. Both test images ship inside the scikit-image package [17] (*skimage.data.hubble_deep_field* and *skimage.data.moon*). The Hubble Deep Field image was captured by NASA and, according to the scikit-image documentation, may be freely used in the public domain; it is the eXtreme Deep Field combining Hubble Space Telescope exposures [24]. The central square of the 872 × 1000 colour image is cropped and resized to 256 × 256; the moon image is resized from 512 × 512 to 256 × 256. A grayscale version of the Hubble image is derived with the standard luminance weights. All other data are generated synthetically by the code itself:

- a low-contrast image (the moon image compressed into the grey range 100–160), 5% salt-and-pepper noise, Gaussian noise of strength $\sigma = 0$ to 20, and a sinusoidal periodic-noise pattern, all produced with a seeded random generator;
- a horizontal illumination gradient multiplied onto the moon image to simulate uneven lighting;
- a Gaussian blur (15 × 15, $\sigma = 3$) and a linear motion blur (21 pixels at 30°), applied in the frequency domain;
- the JPEG luminance and chrominance quantization tables of Annex K of ITU-T T.81 [1], scaled by the quality factor with the same rule that libjpeg uses.

The only external program used as a reference is the JPEG encoder in the Pillow library (Section 5.2). It supplies no data to the pipeline.

2.2 Digital image fundamentals (Unit I)

Sampling is simulated by keeping every k-th pixel (k = 2, 4, 8) and replicating it back to full size; quantization reduces the number of grey levels from 256 to 64, 16, 4 and 2 by uniform binning. These show the two ways in which a continuous scene loses information when it is digitised: spatial resolution and intensity resolution.

2.3 Spatial-domain enhancement (Unit II)

Point operations (negative, log, gamma, contrast stretching) map every pixel independently. Histogram equalization uses the cumulative distribution function of the intensities as the mapping:

$$s = T(r) = \mathrm{round}(\, 255 \cdot (CDF(r) - CDF_{min}) \,/\, (N - CDF_{min}) \,)$$

Histogram specification matches the histogram of an image to that of a reference (here the standard cameraman test image bundled with scikit-image). CLAHE [7, 8] equalizes small tiles and limits the contrast gain with a clip limit. Spatial filters use a 2-D convolution written directly in NumPy: box and Gaussian kernels for smoothing, a median filter for impulse noise, and Laplacian, unsharp-mask and high-boost filters for sharpening.

2.4 Frequency-domain processing (Unit III)

The image is transformed with the 2-D FFT, multiplied by a mask H(u,v) and transformed back. Ideal, Butterworth and Gaussian low-pass masks keep smooth content; high-pass masks are $1 - H_{lp}$. High-frequency emphasis and homomorphic filtering [10] were also implemented. Homomorphic filtering works on the logarithm of the image so that slowly varying illumination and fine reflectance detail can be weighted differently. A Gaussian notch-reject filter removes periodic interference by suppressing the pair of spectral spikes it creates.

2.5 Image restoration (Unit IV)

Degradation is modelled as G = H·F + N, where H is the transform of a Gaussian or motion-blur point spread function and N is additive Gaussian noise. Three restoration filters were implemented:

- **Inverse filter:** $\hat{F} = G / H$, with the frequencies where |H| is below a threshold set to zero. Noise is amplified wherever H is small, so the threshold acts as a low-pass cut-off.
- **Wiener filter:** $\hat{F} = H^{*} \cdot G / (|H|^2 + K)$, where the constant K approximates the noise-to-signal power ratio.
- **Constrained least squares (CLS) [9]:** $\hat{F} = H^{*} \cdot G / (|H|^2 + \gamma |P|^2)$, where P is the Laplacian operator, which adds a smoothness constraint.

2.6 Colour processing and compression (Unit V)

Colour images are converted from RGB to YCbCr because luminance carries most of the visible detail while the two chroma channels can be reduced more aggressively. HSV, CMY and pseudo-colour mapping were also implemented, and the V channel of HSV is used to equalise a colour image without shifting its hues. The codec then processes the image as follows:

- Chroma channels are subsampled by 2 in each direction (4:2:0); pixel values are shifted by −128.
- Every channel is split into 8 × 8 blocks and each block is transformed with the 2-D DCT [2], $F = C \cdot f \cdot C^{T}$, using an orthonormal DCT matrix C that is verified against SciPy.
- Coefficients are divided by the standard quantization table scaled by the quality factor and rounded. This is the only lossy step.
- Each block is scanned in zigzag order. The DC term is coded as a difference from the previous block (category + amplitude bits), and the AC terms are coded as (zero-run, size) symbols with amplitude bits, ZRL and end-of-block symbols.
- Four Huffman codes [3] (DC and AC, for luminance and for chrominance) are built from the symbol frequencies of the image and turned into canonical codes. The decoder performs the exact inverse and is run on the real bit stream to verify the entropy stage.

2.7 Morphology, edges and segmentation (Unit VI)

Sobel and Prewitt gradients are computed with the project's own convolution routine, and Canny edges [5] with scikit-image. Binary dilation, erosion, opening and closing use a 3 × 3 structuring element. Segmentation uses Otsu's threshold [6], which maximises between-class variance, 4-connected region growing from a seed pixel, and K-means clustering [13] on grey levels or RGB values.

2.8 Feature extraction (Unit VII)

Connected regions of the cleaned Otsu mask are described by area, perimeter, centroid, eccentricity and solidity. Texture is described by grey-level co-occurrence statistics [11] (contrast, homogeneity, energy, correlation) and by the entropy of the local binary pattern histogram [12].

2.9 Evaluation metrics

PSNR = $10 \cdot \log_{10}(255^2 / \mathrm{MSE})$ measures pixel error; SSIM [4] compares local luminance, contrast and structure on a 0–1 scale; entropy measures the average information per pixel; compression ratio is the original size in bits divided by the compressed size including the header estimate; the edge F1 score compares two edge maps, optionally with a one-pixel tolerance. For binary maps, the confusion-matrix metrics accuracy, sensitivity, specificity, precision, F1 and Cohen's kappa [14] are defined in Section 5.1.

2.10 Implementation

The code is split into a library module (ipa_core.py: metrics, filters, FFT tools, restoration, colour, codec, morphology) and a runner (run_all.py) that executes the experiments of every unit and writes all figures and tables to the results folder. Eleven built-in tests check the core routines: convolution and the median filter against SciPy, the DCT matrix against SciPy's orthonormal DCT, invertibility of the zigzag scan, Otsu's threshold against scikit-image, the identity frequency filter, monotonicity of the equalization mapping, the prefix-free property of the Huffman code, losslessness of the entropy coder (encode then decode), the YCbCr round trip and the PSNR cap. In the final run 11 of 11 tests passed.

3. Experiment Segment

The experiments are presented in seven parts, one for each group of units, followed by a discussion (Part 8). All figures and tables were generated by the project code.

Part 1: Sampling and quantization

Reducing the spatial resolution makes the lunar image blocky (PSNR 35.9, 31.4 and 28.8 dB for k = 2, 4, 8), and reducing the grey levels produces false contours in smooth regions (PSNR 46.1, 33.5, 19.0 and 7.2 dB for 64, 16, 4 and 2 levels). The contours are clearly visible at 4 levels and below (Figure 1).

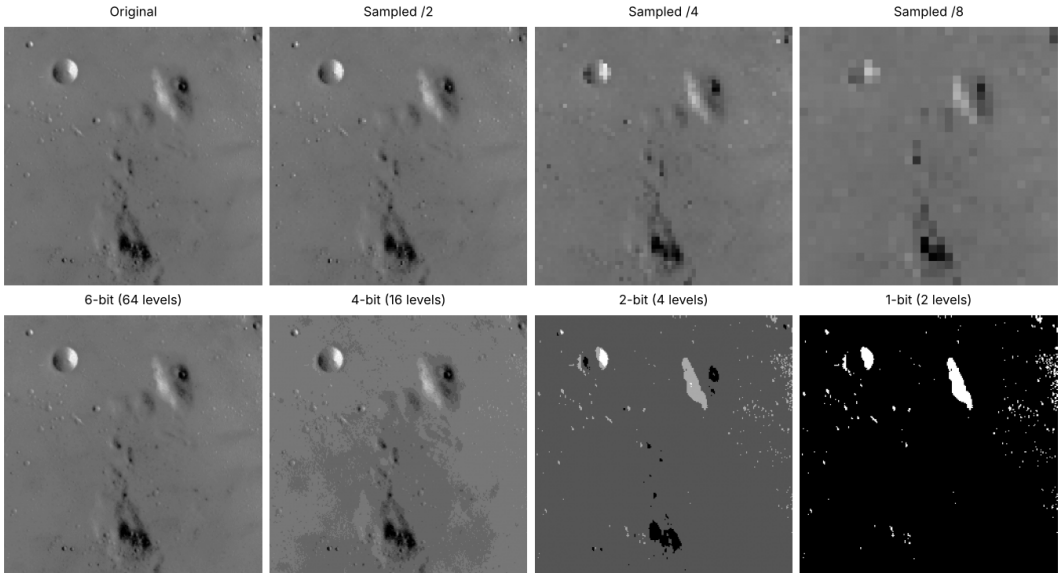


Figure 1: Effect of sampling (top row) and grey-level quantization (bottom row) on the lunar image.

Part 2: Spatial enhancement and denoising

The moon image was compressed into the narrow grey range 100–160 to create a low-contrast input (entropy 2.80 bits/pixel, standard deviation 3.1) and then enhanced with several methods (Figure 2, Table 1). Contrast stretching restores the original almost exactly (41.6 dB against the original) because

it is a reversible linear map. Plain histogram equalization spreads the levels over the full range (standard deviation 74.0) but cannot raise the entropy (2.75 bits/pixel against 2.80), since it is a many-to-one mapping that can only merge intensity levels. CLAHE reached 5.50 bits/pixel by equalizing locally, which brings out fine surface detail; note that this value is higher than that of the original image (4.89) because local equalization also amplifies texture and noise. The log and gamma ($\gamma = 0.6$) transforms do not help here: they lighten the image but even lower its standard deviation (1.3 and 2.6 against 3.1).

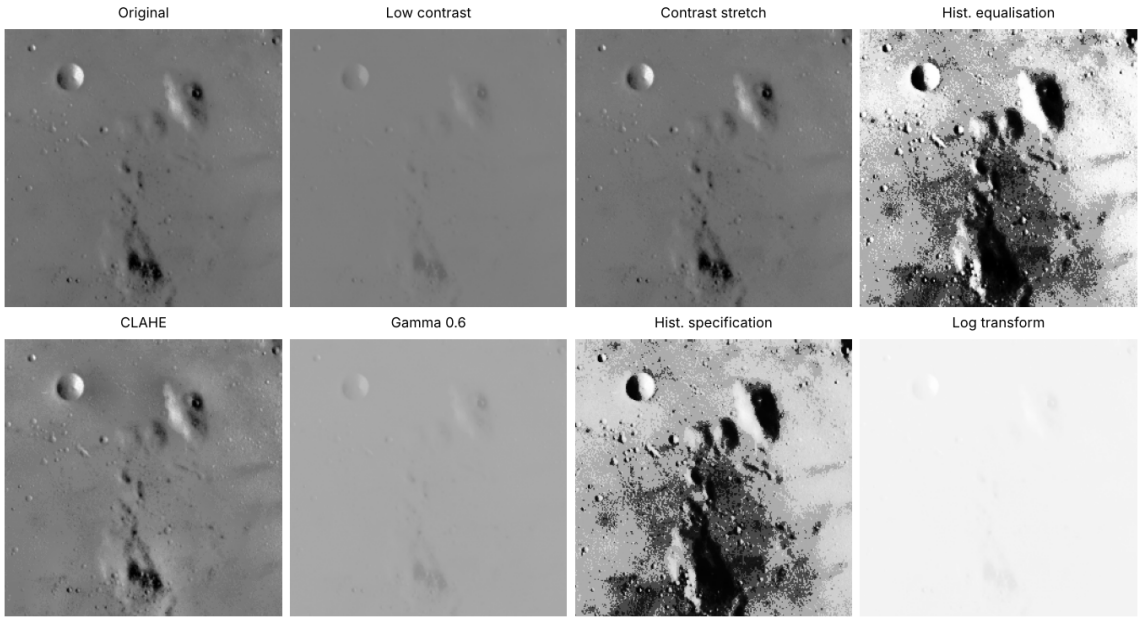


Figure 2: Contrast enhancement of the low-contrast lunar image.

Table 1: Enhancement of the low-contrast image (PSNR is measured against the original moon image and is only meaningful for reversible maps).

Method	Entropy (bits/pixel)	Std. deviation	PSNR vs original (dB)
Low-contrast input	2.80	3.1	23.2
Negative	2.80	3.1	21.0
Log transform	1.59	1.3	5.7
Gamma 0.6	2.52	2.6	13.1
Contrast stretch	2.80	13.4	41.6
Histogram equalisation	2.75	74.0	10.2
Histogram specification	2.74	68.2	11.2
CLAHE	5.50	16.4	24.3
Original moon	4.89	13.1	–

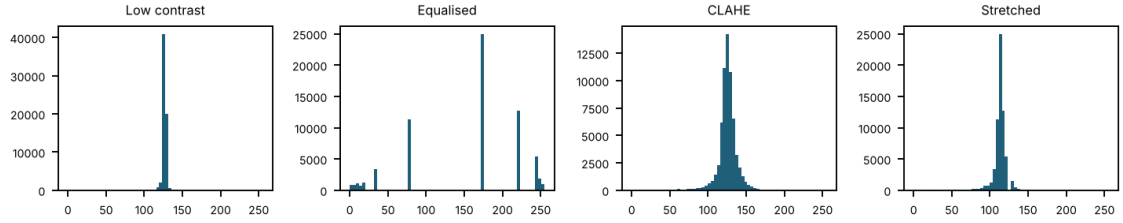


Figure 3: Histograms of the low-contrast input and of the enhanced results.

For 5% salt-and-pepper noise the median filter clearly outperformed the linear filters (Table 2, Figure 4): a median filter discards outliers instead of averaging them into the neighbouring pixels. The result is reversed for Gaussian noise of $\sigma = 20$, where the Gaussian smoothing filter is best, because averaging is the right statistic for zero-mean noise and the median also removes some true texture.

Table 2: Denoising results (PSNR in dB against the clean lunar image).

Method	Salt-and-pepper 5%	Gaussian noise σ = 20
Noisy image	18.90	22.15
Box 3x3	28.04	31.01
Gaussian 5x5	29.21	32.09
Median 3x3	39.55	29.43

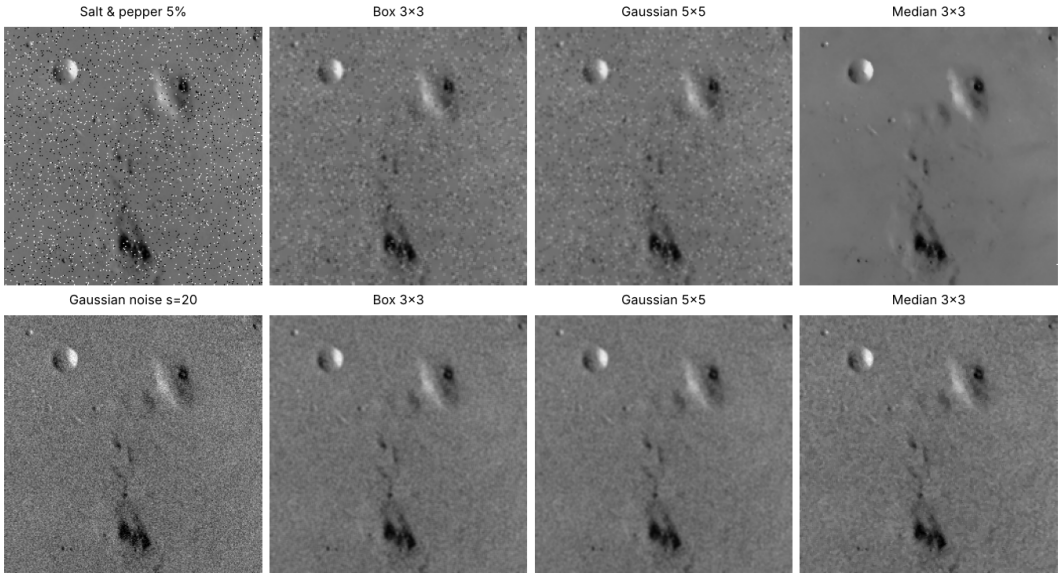


Figure 4: Smoothing of two kinds of noise: impulse noise (top) and Gaussian noise (bottom).

Sharpening with Laplacian, unsharp-mask and high-boost filters increases the variance of the Laplacian (a simple sharpness measure) from 95.9 to 330.2 (unsharp mask), 904.8 (high-boost, k = 2.5) and 2490.2 (Laplacian), at the price of also amplifying noise (Figure 5).

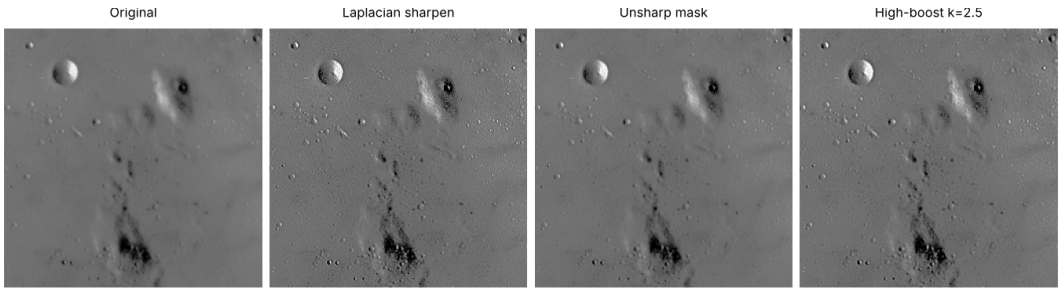


Figure 5: Sharpening filters applied to the lunar image.

Part 3: Frequency-domain filtering

The ideal low-pass filter produces faint ripples around the bright crater and dark pits because its sharp cut-off corresponds to a sinc function in the spatial domain; the Butterworth and Gaussian filters, which roll off smoothly, produce none (Figure 6). Lowering the cut-off D0 increases the blur: the Gaussian filter gives 42.8, 37.7 and 32.3 dB for D0 = 60, 30 and 10 (Table 3). At the same cut-off D0 = 30 the ideal filter (35.9 dB) is below the Butterworth (36.9 dB) and Gaussian (37.7 dB) filters.

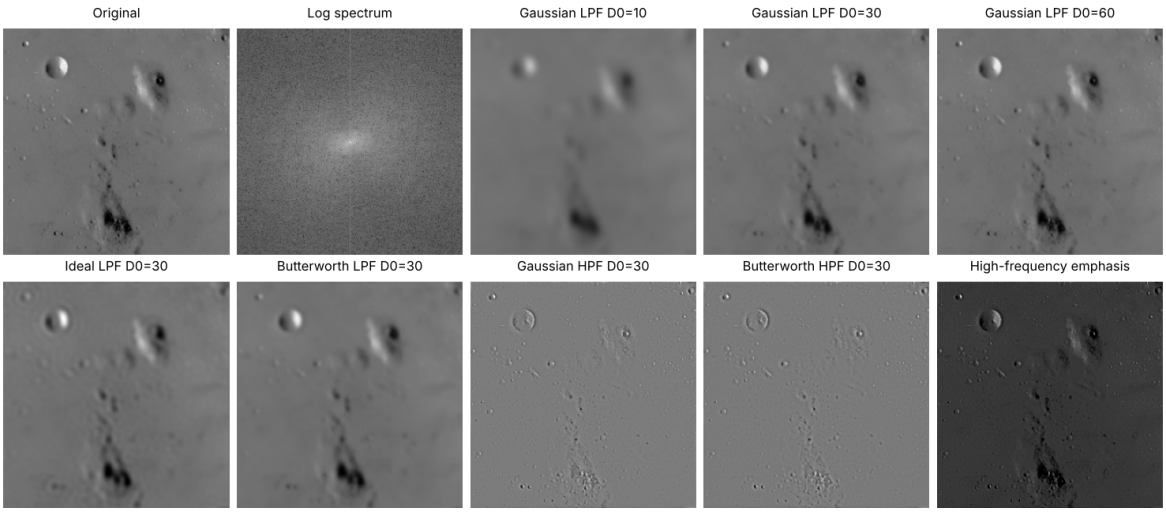


Figure 6: Spectrum, low-pass, high-pass and high-frequency-emphasis filtering of the lunar image.

Table 3: Low-pass filtering of the lunar image (PSNR against the original).

Filter	PSNR (dB)
Gaussian LPF D0=10	32.35
Gaussian LPF D0=30	37.70
Gaussian LPF D0=60	42.77
Ideal LPF D0=30	35.92
Butterworth LPF D0=30	36.94

Two further frequency-domain tools were tested (Figure 7). Homomorphic filtering ($\gamma_L = 0.5$, $\gamma_H = 1.8$, D0 = 30) was applied to an image with a left-to-right illumination gradient; it reduced the variation of the local mean brightness (relative to the mean) from 0.35 to 0.19, flattening the lighting while keeping local detail. A sinusoidal interference pattern of amplitude 35 lowers the PSNR to 20.3 dB and appears as a pair of bright spikes in the spectrum; a Gaussian notch-reject filter on those two spikes raises the PSNR to 48.8 dB.

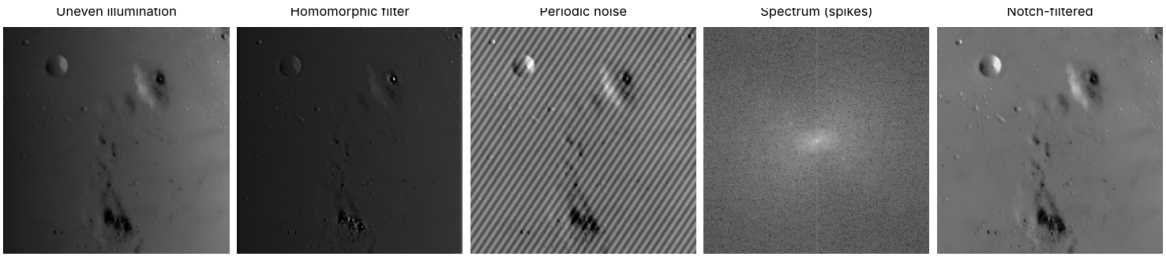


Figure 7: Homomorphic correction of uneven illumination and notch filtering of periodic noise.

Part 4: Image restoration

The lunar image was blurred with a Gaussian point spread function (15 × 15, σ = 3) and noise of increasing strength was added; the degraded image was rounded to 8 bits as a real sensor would do. All tunable filters (inverse threshold, Wiener K, CLS γ) were set with the original image available, so Table 4 shows the best each filter can do. The plain inverse filter with the fixed threshold 10^{-3} performs very badly (12.3 dB without added noise and about 6 dB with it), because even the rounding error of the 8-bit image is divided by values of H as small as 10^{-3}. With a tuned threshold, which turns it into a low-pass cut-off, the inverse filter becomes competitive; at σ = 10 and 20 it even beats the Wiener and CLS filters (30.3 dB against 26.8 and 28.7 dB at σ = 20). CLS is better than Wiener at every noise level, by 0.05 to 2.0 dB.

The last column of Table 4 shows a *blind* Wiener filter whose constant K is set to σ^2/σ_f^2 (noise variance over image variance) instead of being tuned. It is good at low noise but falls to 10.2 dB at σ = 20, even

below the degraded image, because a single constant poorly describes an image spectrum that is far from white. This shows that the K parameter must be estimated carefully in practice.

Table 4: Restoration quality (PSNR in dB) against noise level σ. Best parameters are given under the filter names.

Noise σ	Degraded	Inverse (thr 1e-3)	Inverse (best thr)	Wiener (best K)	CLS (best γ)	Wiener (K = σ^2/σ_f^2)
0	33.84	12.31	36.20 (0.03)	36.45 (0.001)	36.50 (0.003)	31.96
2	33.26	6.65	35.04 (0.1)	35.05 (0.01)	35.46 (0.1)	34.11
5	30.99	6.20	34.06 (0.3)	32.63 (0.03)	34.53 (0.3)	24.55
10	27.08	6.05	33.01 (0.3)	30.56 (0.03)	32.51 (0.3)	15.70
20	21.82	5.98	30.31 (0.3)	26.81 (0.03)	28.68 (0.3)	10.18

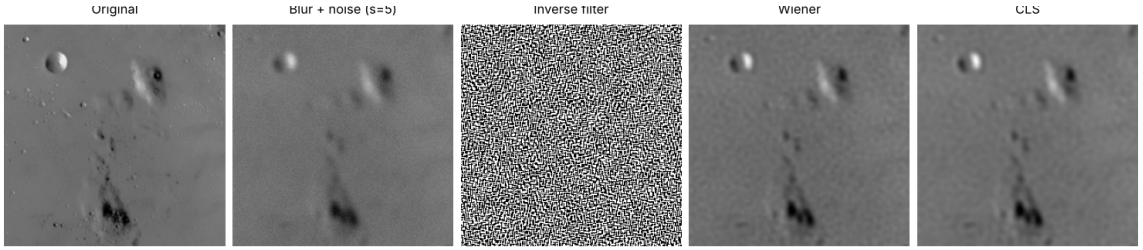


Figure 8: Restoration of a blurred, noisy image (σ = 5).

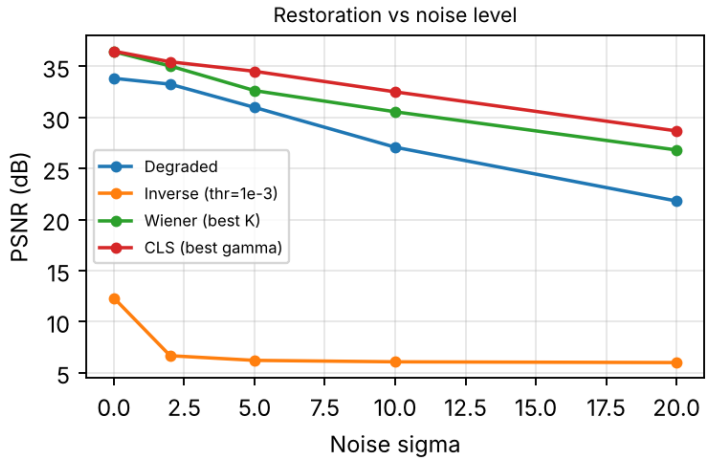


Figure 9: Restoration PSNR against noise level.

The same filters were also tried with a known motion-blur PSF (21 pixels at 30°, noise σ = 1). The inverse filter with threshold 10^{-2} gives 19.0 dB, below the degraded image (30.3 dB) because it leaves a noise pattern, while the Wiener filter with K = 0.01 gives 33.2 dB, an improvement of 2.9 dB, with some ringing near the borders (Figure 10).

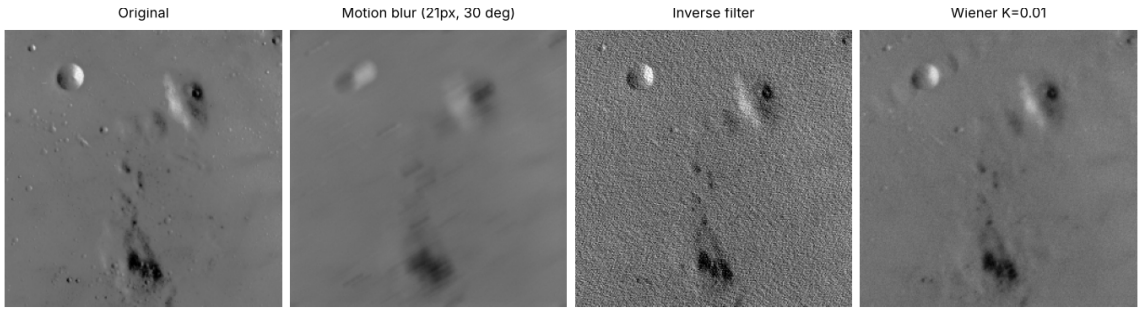


Figure 10: Restoration of motion blur with the inverse and Wiener filters.

Part 5: Colour models and compression

In the YCbCr representation the Y channel holds the structure while Cb and Cr are smooth, which justifies subsampling them (Figure 11). Equalizing only the V channel in HSV reveals the faint galaxies without changing their hue and raises the entropy of the grayscale equivalent from 4.13 to 6.42 bits/pixel; pseudo-colouring the intensity makes brightness differences easy to read.

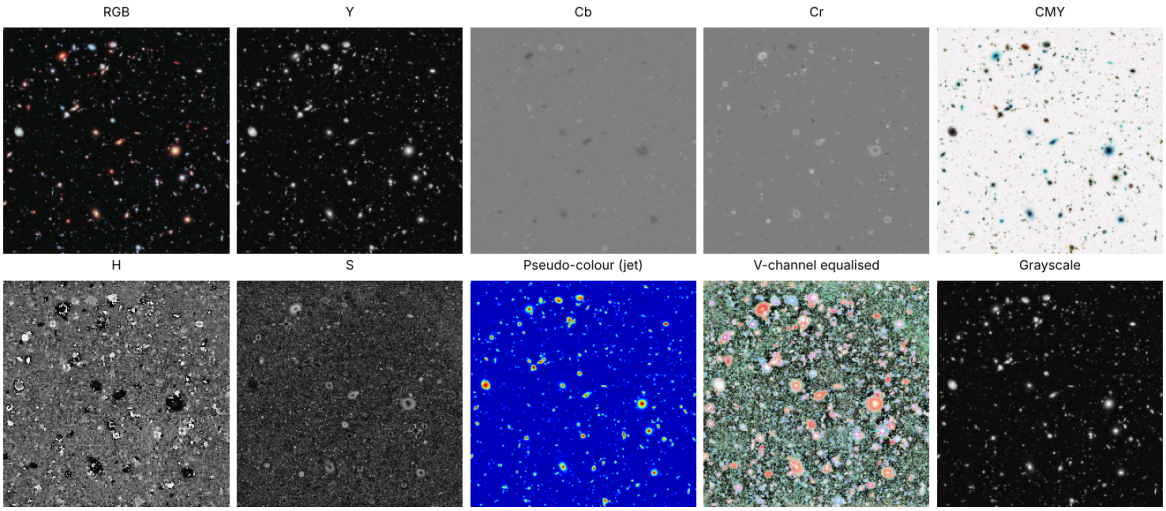


Figure 11: RGB, YCbCr, CMY, HSV, pseudo-colour and V-equalised representations of the Hubble image.

The codec was evaluated at seven quality factors (Table 5). Compression ratios are very high because most of the image is nearly black, so almost every AC coefficient quantizes to zero: the ratio is 151.3:1 at quality 5 and still 9.1:1 at quality 90. The entropy stage was verified on the real bit stream (encode then decode) at qualities 10, 50 and 90 and was lossless in every case. Compression ratio and PSNR trade off smoothly (Figure 14); blocking artifacts are visible at quality 10 (Figure 13) and become hard to see from quality 50 upward (Figure 12).

Table 5: Codec performance on the 256 × 256 colour Hubble image.

Quality	Bits/pixel	Ratio	PSNR (dB)	SSIM	Pillow ratio (default)	Pillow ratio (optimised)
5	0.159	151.3	24.41	0.558	85.7	145.1
10	0.272	88.3	26.45	0.647	63.1	86.1
20	0.472	50.8	28.33	0.749	42.1	49.6
30	0.643	37.3	29.27	0.788	32.8	36.7
50	0.939	25.5	30.50	0.829	23.6	25.2
70	1.323	18.1	31.79	0.866	17.3	17.9
90	2.627	9.1	35.04	0.929	8.9	9.0

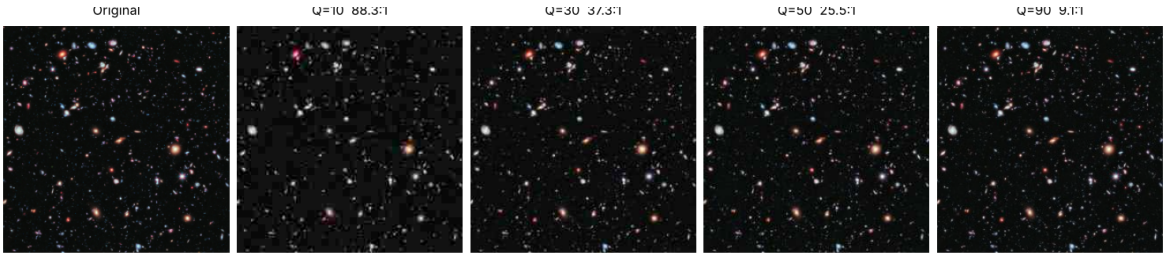


Figure 12: Decoded images at quality 10, 30, 50 and 90.

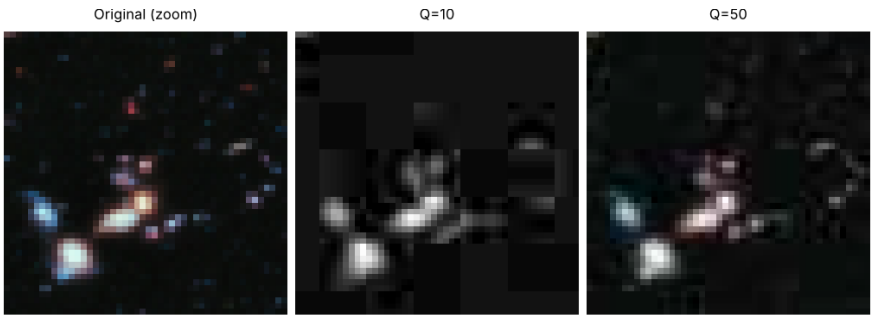


Figure 13: Blocking artifacts: zoom on a 48 × 48 region at quality 10 and 50.

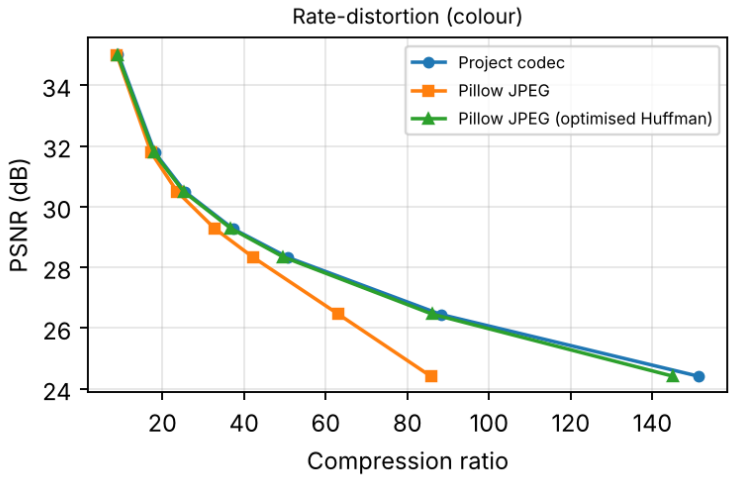


Figure 14: Rate-distortion curves of the project codec and Pillow JPEG.

Chroma subsampling raised the compression ratio at quality 50 from 22.5 to 25.5 at a cost of 0.43 dB PSNR. For the grayscale version of the image the entropy of the raw pixels is 4.13 bits/pixel, while the codec reaches 0.86 bits/pixel for the payload and 0.88 bits/pixel with the header estimate included at quality 50.

Part 6: Edges, segmentation and morphology

Sobel and Prewitt gradient magnitudes (thresholded with Otsu's method) mark 4.4% and 4.5% of the lunar pixels as edges and give almost identical maps, while Canny ($\sigma = 1.5$, hysteresis on the 80th and 92nd percentiles) gives thinner, connected edges at 2.9% of the pixels (Figure 15). To measure how compression affects analysis, Canny edges detected on decoded lunar images were compared with those of the original image (Section 5.5).

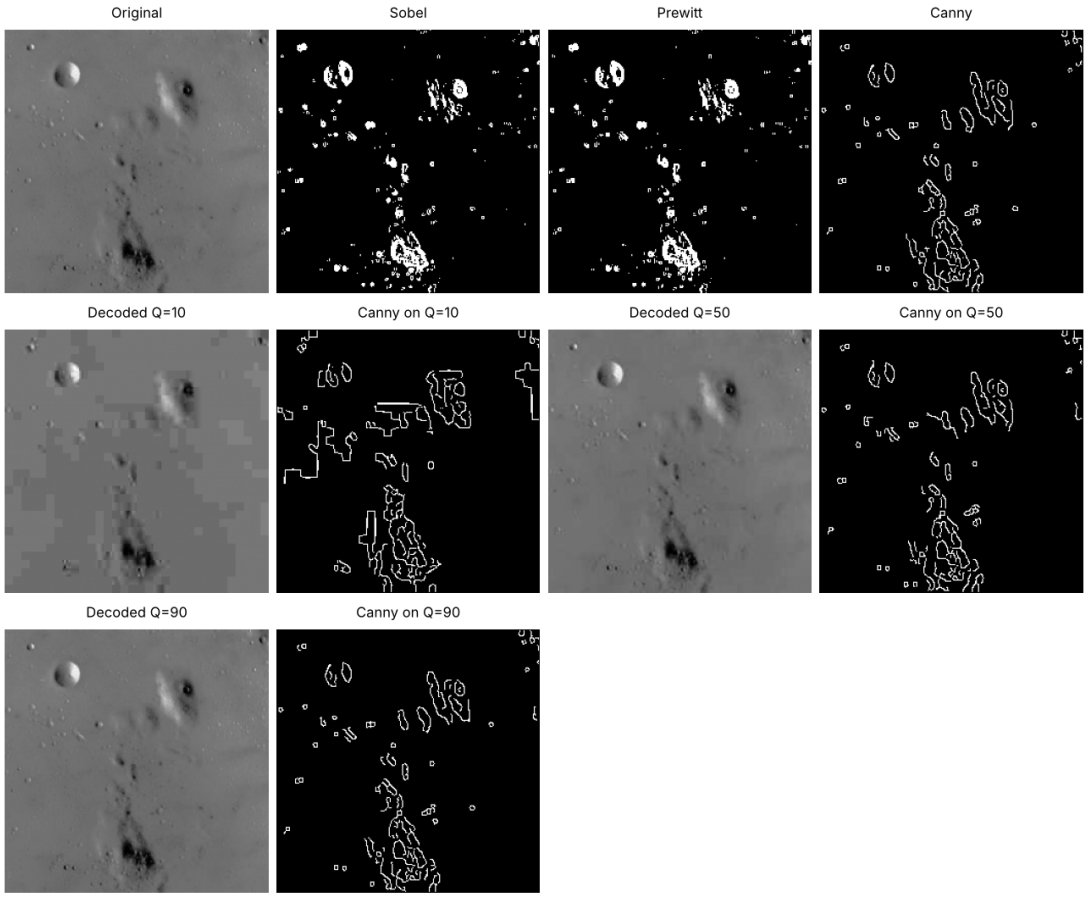


Figure 15: Edge detection on the original and on decoded lunar images.

Otsu's method gave a threshold of 71 for the grayscale Hubble image and marks 3.4% of the pixels as objects, split into 420 connected components. Opening followed by closing removed isolated noise pixels and small faint sources and left 38 components; dilation grows the objects and erosion removes all but the largest ones. Region growing from a sky pixel (tolerance 12 grey levels) selected 89.0% of the image as background. K-means separated the moon image into three grey clusters (centres 61.4, 113.2, 169.7) and the colour Hubble image into four clusters (Figure 16).

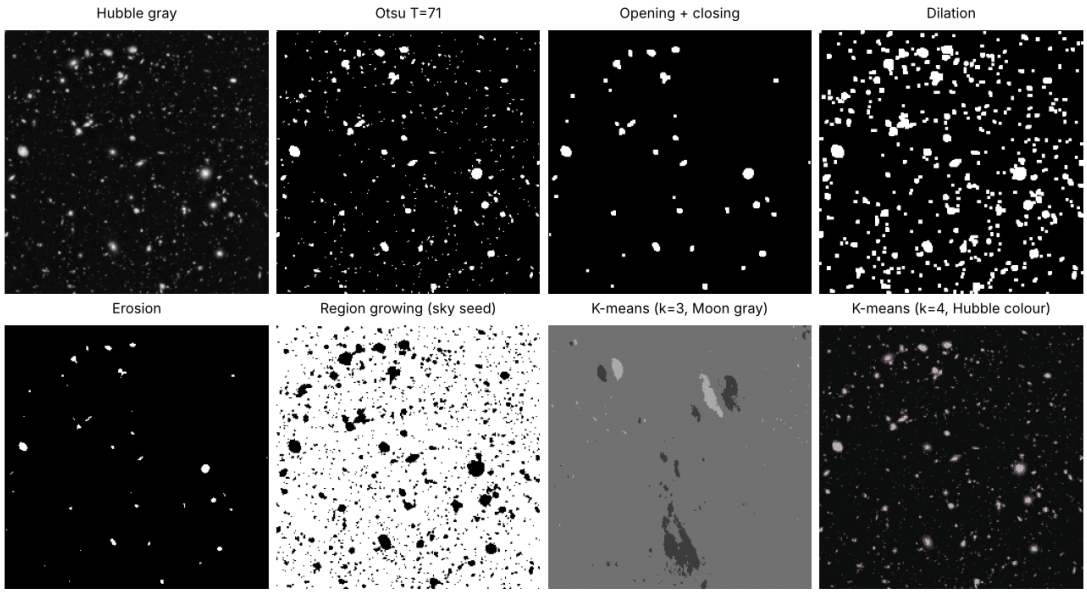


Figure 16: Thresholding, morphology, region growing and K-means.

Part 7: Feature extraction and application: counting objects in the Hubble field

The largest regions of the cleaned Otsu mask were described by shape features (Table 6). The two images also have very different textures (Table 7): the Hubble field has a GLCM contrast of 18.1, ten times that of the smoother lunar surface (1.75), and lower correlation (0.74 against 0.92), because isolated bright objects on a dark background make neighbouring pixels less alike. The LBP entropies are nearly equal.

Table 6: Shape features of the six largest cleaned objects in the Hubble image (pixel units).

#	Area	Perimeter	Centroid (row, col)	Eccentricity	Solidity
1	95	33.6	(139, 194)	0.37	0.95
2	93	34.1	(117, 17)	0.63	0.94
3	69	33.9	(45, 113)	0.49	0.84
4	54	26.5	(33, 67)	0.65	0.92
5	53	25.3	(210, 105)	0.77	0.93
6	50	23.9	(22, 100)	0.71	0.98

Table 7: Texture features (GLCM at distance 1 averaged over four directions, 64 grey levels; LBP entropy).

Image	Contrast	Homogeneity	Energy	Correlation	LBP entropy
Moon surface	1.75	0.749	0.304	0.920	3.21
Hubble (gray)	18.10	0.644	0.427	0.736	3.22

Application. To show a practical use of the whole pipeline, bright objects in the grayscale Hubble image were detected with a classical procedure: Gaussian smoothing ($\sigma = 1$), a global threshold at the mean plus two standard deviations (56.5 grey levels), connected-component labelling and removal of components smaller than 3 pixels. This finds **121 objects** with a mean area of 19.6 pixels and a largest area of 131 pixels; 58 of them are nearly round (eccentricity below 0.6) and 63 are elongated. After a gamma transform ($\gamma = 0.5$) that lifts the dark end of the histogram and a lower threshold (mean plus one standard deviation, 86.3 grey levels) the same procedure finds **254 objects** (Figure 17). The extra detections are mostly faint galaxies that were hidden in the dark background, but at this threshold some noise blobs are certainly counted as well; the counts are the number of bright blobs, not a validated galaxy catalogue. The size distribution is strongly skewed towards small objects (Figure 18), as expected for a deep field where faint, distant galaxies outnumber bright, nearby ones.

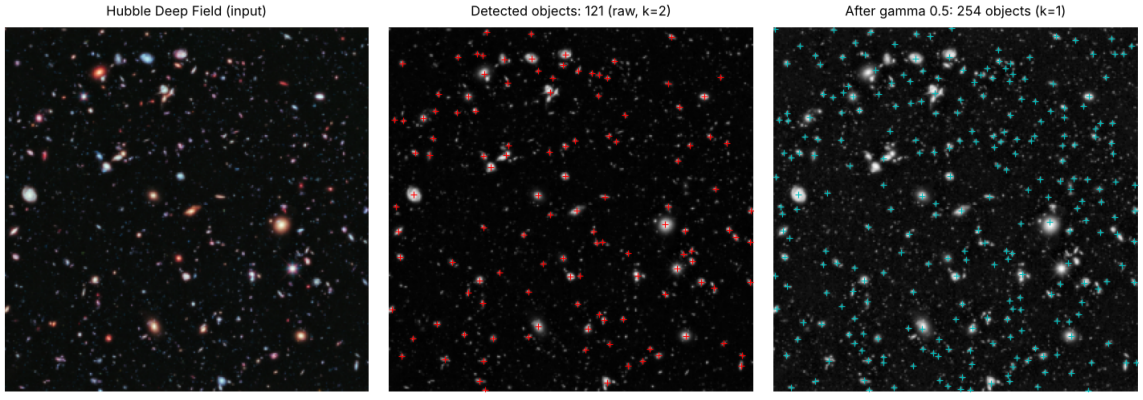


Figure 17: Counting objects in the Hubble Deep Field without and with gamma enhancement.

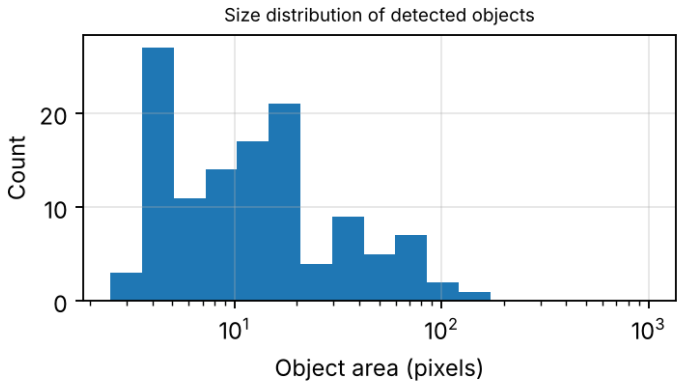


Figure 18: Size distribution of the detected objects (log scale).

Part 8: Discussion

- The results illustrate a recurring theme: methods that are perfect on ideal data (the plain inverse filter, the ideal low-pass filter) are fragile in practice, and regularised or smooth alternatives (Wiener, CLS, Butterworth, Gaussian) are more robust.
- There is no single best denoiser. The median filter wins by about 10 dB on impulse noise, but the Gaussian filter wins by about 2.7 dB on Gaussian noise. The noise model decides the filter.
- Lossy compression discards what a viewer notices least, but its cost is not uniform: PSNR changes slowly at high quality (35.0 dB at quality 90, 30.5 dB at 50) and falls quickly below quality 20.
- Edge-based analysis is more tolerant of compression than a pixel-wise comparison suggests (Section 5.5), which matters for pipelines that compress images before running vision algorithms.
- Enhancement changes what an analysis can find: a simple gamma transform roughly doubled the number of detected objects in the deep field, so the order of the pipeline stages matters.

4. Literature Referred or Reviewed

This section summarises the literature read for each stage of the project. The standard textbooks listed in the Bibliography were the main source for the course material, and the papers below were consulted for the specific techniques that were implemented.

4.1 Enhancement

Histogram equalization is the classical global contrast method. Pizer et al. [8] introduced adaptive histogram equalization, which computes the mapping from a local neighbourhood, and Zuiderveld [7] added the contrast limit that gives CLAHE and prevents noise amplification in flat regions; this is why CLAHE could bring out surface detail in Part 2 where plain equalization could not. Homomorphic filtering [10] models an image as the product of illumination and reflectance and separates them in the logarithmic domain.

4.2 Restoration

Inverse filtering is the direct solution of the degradation model but is ill-posed when the blur transfer function has small values. The Wiener filter, derived by minimising the mean-square error between the estimate and the true image, adds a noise-to-signal term that regularises the division. Hunt [9] formulated restoration as constrained least squares, which uses a smoothness constraint instead of a known noise spectrum. Iterative methods such as Richardson–Lucy deconvolution [21, 22] assume Poisson noise and are widely used in astronomy, including for Hubble images, and in microscopy. When the blur is unknown, blind deconvolution must estimate it as well; Fergus et al. [23] showed this for camera shake.

4.3 Compression

The DCT was proposed by Ahmed, Natarajan and Rao [2] and is close to the optimal Karhunen–Loève transform for highly correlated image data, which explains its use in JPEG. Wallace [1] describes the baseline JPEG standard that this project's codec follows: level shift, 8 × 8 DCT, quantization, zigzag scan, differential DC coding, run-length coding of AC coefficients and Huffman coding [3]. Arithmetic coding [20] can replace Huffman coding and lowers the bit rate at the same quality. JPEG 2000 [19] replaces the block DCT with a wavelet transform, which removes the blocking artifacts visible at low quality in the zoomed figure of Part 5.

4.4 Segmentation, edges and features

Canny [5] defined edge detection as an optimisation of detection, localisation and single response and introduced non-maximum suppression and hysteresis thresholding. Otsu's method [6] selects the threshold that maximises between-class variance. K-means clustering [13] partitions pixel values into k groups by minimising the within-cluster squared distance. For texture, Haralick et al. [11] defined statistics of the grey-level co-occurrence matrix, and Ojala et al. [12] introduced the local binary pattern.

4.5 Quality metrics and the data

PSNR is simple but correlates imperfectly with perceived quality; Wang and Bovik [15] and Huynh-Thu and Ghanbari [16] discuss its scope and limits. SSIM [4] compares local structure and was designed to match human judgement better. For binary maps, Cohen's kappa [14] corrects the agreement between two labellings for the agreement expected by chance, which makes it more informative than accuracy when one class (non-edge pixels) dominates. The Hubble eXtreme Deep Field itself, combining about ten years of Hubble exposures, is described by Illingworth et al. [24].

4.6 Summary of the reviewed work

Table 8: Summary of the literature reviewed and how it is used in this project.

Reference	Contribution	Used in this project for
Ahmed et al. [2]	Discrete cosine transform	8 × 8 block transform in the codec
Huffman [3]	Minimum-redundancy codes	Entropy coding of DC and AC symbols
Wallace [1]	Baseline JPEG standard	Overall codec design and quantization tables
Pizer et al. [8]; Zuiderveld [7]	Adaptive and contrast-limited equalization	CLAHE enhancement (Part 2)
Oppenheim et al. [10]	Homomorphic filtering	Illumination correction (Part 3)
Hunt [9]	Constrained least-squares restoration	CLS filter (Part 4)
Canny [5]; Otsu [6]	Edge detection; thresholding	Edges and masks (Part 6)
Haralick et al. [11]; Ojala et al. [12]	GLCM and LBP texture	Feature extraction (Part 7)
Wang et al. [4]	Structural similarity index	SSIM metric
Cohen [14]	Chance-corrected agreement	Kappa score in Section 5
Illingworth et al. [24]	Hubble eXtreme Deep Field	Test image and application (Part 7)

5. Comparison with Others

This section compares the results with (a) a reference JPEG encoder and (b) the standard classical alternatives within the project. Every number quoted comes from the project's own experiments or from the reference encoder run on the same image and settings.

5.1 Metrics used

For image fidelity, PSNR, SSIM and compression ratio are used. For binary maps (an edge map or a segmentation mask compared with a reference), each pixel is classified against the reference, giving

true positives (TP), true negatives (TN), false positives (FP) and false negatives (FN). The following metrics are computed from these counts:

$$Accuracy = (TP + TN) / (TP + TN + FP + FN)$$

$$Sensitivity\ (recall) = TP / (TP + FN)\ Specificity = TN / (TN + FP)$$

$$Precision = TP / (TP + FP)\ F1 = 2 \cdot Precision \cdot Recall / (Precision + Recall)$$

$$Cohen's\ kappa = (p_o - p_e) / (1 - p_e),\ p_o = Accuracy,\ p_e = chance\ agreement\ from\ row\ and\ column\ totals$$

Accuracy alone is misleading for edge maps because only about 3% of the pixels are edges, so an empty map would already score about 97%. Sensitivity, F1 and kappa are therefore the more informative scores.

5.2 Compression: the codec against the Pillow JPEG encoder

Table 9 compares the project codec with the Pillow JPEG encoder on the same 256 × 256 colour image, both with 4:2:0 chroma subsampling. Pillow was run twice: with its default (fixed, standard) Huffman tables and with per-image optimised tables, which is the fairer match for the codec because the codec also builds its tables from the image. Figure 19 plots the compression ratios.

Table 9: Project codec against Pillow JPEG on the 256 × 256 colour Hubble image.

Quality	Bits/pixel codec / Pillow	Ratio codec / Pillow / Pillow-opt.	Ratio diff. vs default	Ratio diff. vs optimised	PSNR codec / Pillow (dB)	SSIM codec / Pillow
5	0.159 / 0.280	151.3 / 85.7 / 145.1	+76.5%	+4.3%	24.41 / 24.41	0.558 / 0.559
10	0.272 / 0.380	88.3 / 63.1 / 86.1	+39.9%	+2.5%	26.45 / 26.46	0.647 / 0.647
20	0.472 / 0.570	50.8 / 42.1 / 49.6	+20.6%	+2.4%	28.33 / 28.34	0.749 / 0.749
30	0.643 / 0.732	37.3 / 32.8 / 36.7	+13.9%	+1.6%	29.27 / 29.29	0.788 / 0.788
50	0.939 / 1.017	25.5 / 23.6 / 25.2	+8.2%	+1.4%	30.50 / 30.50	0.829 / 0.830
70	1.323 / 1.391	18.1 / 17.3 / 17.9	+5.1%	+1.3%	31.79 / 31.79	0.866 / 0.865
90	2.627 / 2.709	9.1 / 8.9 / 9.0	+3.1%	+1.3%	35.04 / 35.01	0.929 / 0.928

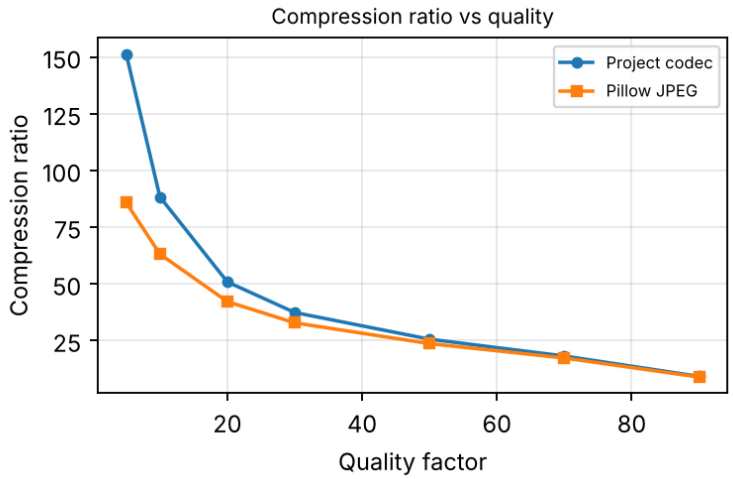


Figure 19: Compression ratio of the project codec and Pillow JPEG against quality factor.

The codec is 3% to 76% better than Pillow's default encoder in compression ratio at every quality setting, and +1.3% to +4.3% against Pillow with optimised tables. The PSNR is within 0.03 dB and the SSIM within 0.002 of the encoder everywhere, as expected, because both use the same transform, quantization tables and subsampling; the differences come only from the entropy coder and from the header. The advantage over the default encoder is therefore mostly due to the per-image Huffman tables, which suit this unusual mostly-black image much better than the generic standard tables,

especially at low quality. The comparison with the optimised encoder is the more informative one and shows that the project codec is practically at the level of a real encoder. Two caveats apply: the codec's output is not a standard JPEG file, and its header size is estimated rather than written, so a few bytes of difference in either direction are possible.

5.3 Denoising: median against linear filters

Against the noisy image at 18.9 dB, the box filter gains 9.1 dB, the Gaussian filter 10.3 dB and the median filter 20.6 dB on impulse noise. The median filter is therefore 10.3 dB better than the best linear filter, because it removes the outliers instead of spreading them. On Gaussian noise ($\sigma = 20$) the order reverses: the Gaussian filter gains 9.9 dB and the median 7.3 dB, so the Gaussian filter is 2.7 dB better.

5.4 Restoration: inverse, Wiener and CLS filters

Table 10: PSNR gain (dB) over the degraded image at each noise level; the last row gives the mean absolute PSNR.

Noise σ	Inverse (thr 1e-3)	Inverse (best thr)	Wiener	CLS
0	-21.52	+2.36	+2.61	+2.66
2	-26.61	+1.78	+1.78	+2.19
5	-24.79	+3.07	+1.65	+3.54
10	-21.03	+5.93	+3.47	+5.43
20	-15.84	+8.49	+5.00	+6.87
Mean PSNR	7.44	33.72	32.30	33.53

The Wiener and CLS filters improve the degraded image at every noise level, by 1.6 to 5.0 dB for Wiener and 2.2 to 6.9 dB for CLS, while the plain inverse filter loses 16 to 27 dB at every noise level because it already fails on the rounding error of the 8-bit image. Their mean PSNR over the five noise levels is 32.3 and 33.5 dB, against 7.4 dB for the plain inverse filter. The truncated inverse filter with a tuned threshold has the highest mean (33.7 dB), which is an honest and slightly surprising outcome: for a Gaussian blur, whose transform decays smoothly, truncating the inverse is already a good regulariser. All three tuned filters used a parameter chosen with the original image available, so these numbers show the best the filters can do and not what a blind user would obtain; the blind Wiener estimate of Table 4 shows how much a poor parameter costs.

5.5 Edge preservation after compression (all metrics)

The Canny edge map of the original grayscale lunar image was compared with the edge map of the same image after a round trip through the project codec and, for reference, through the Pillow JPEG encoder at the same quality. Table 11 reports the exact pixel-wise agreement, and the last column the F1 score with a one-pixel tolerance, which treats a displaced edge as correct.

Table 11: Canny-edge agreement after compression (lunar image, 256 × 256).

Quality	Codec	PSNR (dB)	Accuracy	Sensitivity	Specificity	Precision	F1	Kappa	F1 (±1 px)
10	Project	34.20	0.957	0.397	0.974	0.312	0.349	0.327	0.639
10	Pillow	34.20	0.957	0.394	0.974	0.311	0.348	0.326	0.639
50	Project	39.64	0.986	0.761	0.993	0.752	0.757	0.749	0.929
50	Pillow	39.64	0.986	0.776	0.993	0.762	0.769	0.762	0.934
90	Project	43.93	0.996	0.928	0.998	0.935	0.932	0.929	0.980
90	Pillow	43.92	0.996	0.924	0.998	0.937	0.931	0.928	0.978

Edge agreement rises steadily with quality. For the project codec at quality 10 the sensitivity is 0.397 and kappa 0.327, so about 60% of the original edge pixels are lost or displaced; at quality 90 the sensitivity is 0.928 and kappa 0.929. Specificity is at least 0.974 at every quality, so very few false edges are created. Accuracy is above 0.96 even at quality 10, which shows again why accuracy alone is a weak measure. Once a one-pixel tolerance is allowed, the F1 score rises from 0.349 to 0.639 at

quality 10, which shows that much of the apparent loss at low quality is a small displacement of edges and not their removal. At every quality the project codec and the Pillow encoder agree to within 0.015 on every pixel-wise score and in PSNR to within 0.02 dB, so the codec costs no edge quality compared with a real encoder.

6. Conclusion

The project covers all seven units of the course in one reproducible pipeline and confirms the expected behaviour of each technique with measurements on real astronomical and lunar images. The from-scratch codec reached 25.5:1 at 30.5 dB on the Hubble image at quality 50, within 1.4% of a real JPEG encoder with optimised tables and about 8% better than the default encoder, with a decoder verified to be lossless on the entropy stage. The median filter gained 20.6 dB on impulse noise while the Gaussian filter was best on Gaussian noise; the regularised Wiener and CLS filters and the truncated inverse filter restored blurred, noisy images where the plain inverse filter failed; a notch filter removed periodic interference (20.3 to 48.8 dB); and Canny edges survived heavy compression (F1 with tolerance 0.64 even at quality 10). A simple gamma enhancement more than doubled the number of objects an automatic detector finds in the deep field (121 to 254), showing why enhancement matters before analysis.

Limitations: (1) the codec writes a custom bit stream, not a standards-compliant JPEG file, and does not limit Huffman code lengths to 16 bits; (2) the header size is estimated; (3) the Wiener K, CLS γ and inverse-filter threshold were selected by comparison with the original image, which is not possible in practice; (4) restoration assumes a known PSF and circular convolution; (5) only two test images were used, and the Hubble image is atypical (mostly black), which inflates compression ratios; (6) the classification metrics compare the Canny edges of decoded images with the edges of their own originals and not with human-annotated edges; (7) the object counts are bright-blob counts from a global threshold and were not checked against an astronomical catalogue.

7. Future Work (Future Scope)

- **Standards-compliant JPEG output.** Limit Huffman code lengths to 16 bits, write the real marker segments and compare the byte-exact file size with Pillow; add arithmetic coding [20] to compress further.
- **Wavelet-based comparison.** Implement a wavelet codec in the manner of JPEG 2000 [19] and add it to the same rate-distortion plot to remove the blocking artifacts seen at low quality.
- **Blind and iterative restoration.** Estimate the point spread function and the noise variance from the image [23], set K from the estimate, and add Richardson–Lucy deconvolution [21, 22], which is the standard method for telescope images.
- **Better object detection.** Replace the global threshold with local background estimation and multi-scale detection, and validate the counts against a published Hubble catalogue.
- **A larger test set.** Repeat the experiments on benchmark sets (Kodak images for compression, BSDS500 for edges) and report means and standard deviations instead of single-image values.
- **Deblocking.** Add a post-processing filter to reduce blocking at low quality and measure its effect on edge detection and segmentation.

8. Code Link

The source code of this project consists of two Python files, *ipa_core.py* (library) and *run_all.py* (experiment runner), supplied together with this report. Running *python run_all.py* regenerates every figure and number in the report in about ten seconds.

GitHub repository: _____ (add the link after uploading the code)

9. Bibliography

Books, standards and documentation that were studied but are not cited by number in the text:

1. R. C. Gonzalez and R. E. Woods, *Digital Image Processing*, 4th ed. New York, NY, USA: Pearson Education, 2018.
2. A. K. Jain, *Fundamentals of Digital Image Processing*. Englewood Cliffs, NJ, USA: Prentice Hall, 1989.
3. W. B. Pennebaker and J. L. Mitchell, *JPEG: Still Image Data Compression Standard*. New York, NY, USA: Van Nostrand Reinhold, 1993.
4. D. S. Taubman and M. W. Marcellin, *JPEG2000: Image Compression Fundamentals, Standards and Practice*. Boston, MA, USA: Kluwer Academic, 2002.
5. K. Sayood, *Introduction to Data Compression*, 5th ed. Cambridge, MA, USA: Morgan Kaufmann, 2017.
6. N. Wiener, *Extrapolation, Interpolation, and Smoothing of Stationary Time Series*. Cambridge, MA, USA: MIT Press, 1949.
7. ITU-T Recommendation T.81, *Information technology: Digital compression and coding of continuous-tone still images (JPEG)*, 1992.
8. scikit-image documentation, https://scikit-image.org/docs/stable/ (image processing functions and bundled test images).
9. NumPy, SciPy, Matplotlib, scikit-learn and Pillow online documentation (array operations, FFT and DCT routines, K-means, plotting and the reference JPEG encoder).

10. References

[1] G. K. Wallace, “The JPEG still picture compression standard,” *Commun. ACM*, vol. 34, no. 4, pp. 30–44, 1991.

[2] N. Ahmed, T. Natarajan, and K. R. Rao, “Discrete cosine transform,” *IEEE Trans. Comput.*, vol. C-23, no. 1, pp. 90–93, 1974.

[3] D. A. Huffman, “A method for the construction of minimum-redundancy codes,” *Proc. IRE*, vol. 40, no. 9, pp. 1098–1101, 1952.

[4] Z. Wang, A. C. Bovik, H. R. Sheikh, and E. P. Simoncelli, “Image quality assessment: from error visibility to structural similarity,” *IEEE Trans. Image Process.*, vol. 13, no. 4, pp. 600–612, 2004.

[5] J. Canny, “A computational approach to edge detection,” *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. PAMI-8, no. 6, pp. 679–698, 1986.

[6] N. Otsu, “A threshold selection method from gray-level histograms,” *IEEE Trans. Syst., Man, Cybern.*, vol. 9, no. 1, pp. 62–66, 1979.

[7] K. Zuiderveld, “Contrast limited adaptive histogram equalization,” in *Graphics Gems IV*. San Diego, CA, USA: Academic Press, 1994, pp. 474–485.

[8] S. M. Pizer *et al.*, “Adaptive histogram equalization and its variations,” *Comput. Vis., Graph., Image Process.*, vol. 39, no. 3, pp. 355–368, 1987.

[9] B. R. Hunt, “The application of constrained least squares estimation to image restoration by digital computer,” *IEEE Trans. Comput.*, vol. C-22, no. 9, pp. 805–812, 1973.

[10] A. V. Oppenheim, R. W. Schafer, and T. G. Stockham, “Nonlinear filtering of multiplied and convolved signals,” *Proc. IEEE*, vol. 56, no. 8, pp. 1264–1291, 1968.

[11] R. M. Haralick, K. Shanmugam, and I. Dinstein, “Textural features for image classification,” *IEEE Trans. Syst., Man, Cybern.*, vol. SMC-3, no. 6, pp. 610–621, 1973.

[12] T. Ojala, M. Pietikäinen, and T. Mäenpää, “Multiresolution gray-scale and rotation invariant texture classification with local binary patterns,” *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 24, no. 7, pp. 971–987, 2002.

[13] S. P. Lloyd, “Least squares quantization in PCM,” *IEEE Trans. Inf. Theory*, vol. 28, no. 2, pp. 129–137, 1982.

[14] J. Cohen, “A coefficient of agreement for nominal scales,” *Educ. Psychol. Meas.*, vol. 20, no. 1, pp. 37–46, 1960.

[15] Z. Wang and A. C. Bovik, “Mean squared error: love it or leave it?,” *IEEE Signal Process. Mag.*, vol. 26, no. 1, pp. 98–117, 2009.

[16] Q. Huynh-Thu and M. Ghanbari, “Scope of validity of PSNR in image/video quality assessment,” *Electron. Lett.*, vol. 44, no. 13, pp. 800–801, 2008.

[17] S. van der Walt *et al.*, “scikit-image: image processing in Python,” *PeerJ*, vol. 2, p. e453, 2014.

[18] C. R. Harris *et al.*, “Array programming with NumPy,” *Nature*, vol. 585, pp. 357–362, 2020.

[19] A. Skodras, C. Christopoulos, and T. Ebrahimi, “The JPEG 2000 still image compression standard,” *IEEE Signal Process. Mag.*, vol. 18, no. 5, pp. 36–58, 2001.

[20] I. H. Witten, R. M. Neal, and J. G. Cleary, “Arithmetic coding for data compression,” *Commun. ACM*, vol. 30, no. 6, pp. 520–540, 1987.

[21] W. H. Richardson, “Bayesian-based iterative method of image restoration,” *J. Opt. Soc. Am.*, vol. 62, no. 1, pp. 55–59, 1972.

[22] L. B. Lucy, “An iterative technique for the rectification of observed distributions,” *Astron. J.*, vol. 79, no. 6, pp. 745–754, 1974.

[23] R. Fergus, B. Singh, A. Hertzmann, S. T. Roweis, and W. T. Freeman, “Removing camera shake from a single photograph,” *ACM Trans. Graph.*, vol. 25, no. 3, pp. 787–794, 2006.

[24] G. D. Illingworth *et al.*, “The HST eXtreme Deep Field (XDF): Combining all ACS and WFC3/IR data on the HUDF region into the deepest field ever,” *Astrophys. J. Suppl. Ser.*, vol. 209, no. 1, art. 6, 2013.

Thank You